UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE LA COSTA

Actividad de Investigación

Pruebas No Funcionales, Registro y Seguimiento de Defectos, y Pruebas de Regresión

Materia: 2026 B2 Laboratorio Abierto, Pruebas
Profesor: José Francisco Reinaga Camacho
Fecha: 14 de octubre de 2026

Integrantes:

222967908 — Corona Méndez Jonathan Ismael
222967843 — González Crisanto Ana Karen
222967835 — Herrera Galindo Brithany Juliette

Puerto Vallarta, Jalisco — 2026

Objetivo

Comprender la importancia que tienen las pruebas no funcionales, la forma correcta de registrar y dar seguimiento a los defectos o errores encontrados en el software, y cómo se aplican las pruebas de regresión para asegurar que el sistema se mantenga estable a lo largo del tiempo.

1. Investigación Teórica

1.1. Pruebas No Funcionales

Concepto y propósito: Las pruebas no funcionales son aquellas que se encargan de revisar cómo se comporta el sistema en lugar de qué operaciones hace. Mientras que las pruebas funcionales revisan que un botón guarde los datos o que una suma dé el resultado correcto, las no funcionales evalúan aspectos como la velocidad de respuesta, si la página soporta a muchos usuarios al mismo tiempo, si la información de las personas está bien protegida y si la interfaz es cómoda y fácil de entender.

Su propósito es asegurar que el programa realmente sea útil en la vida real. De nada sirve que un sistema haga los cálculos a la perfección si tarda dos minutos en cargar una pantalla, si se traba cuando entran diez personas a la vez o si cualquier persona puede ver las contraseñas de los demás. Por eso, estas pruebas buscan que la experiencia de uso sea buena, segura y confiable.

Tipos de Pruebas No Funcionales

Existen diferentes tipos de pruebas no funcionales según lo que se necesite evaluar:

- **Rendimiento (Performance):** Revisan qué tan rápido responde la aplicación y cuánta memoria o procesador consume. Aquí entran las pruebas de carga (para ver cómo funciona con el número normal de usuarios), las de estrés (para ver qué pasa si se le exige más de lo normal hasta que falle), las de resistencia (para ver si después de muchas horas encendido no se vuelve lento) y las de picos (para ver cómo reacciona a subidas repentinas de tráfico).
- **Seguridad:** Verifican que el sistema esté protegido contra accesos no autorizados o ataques comunes. Revisan que las contraseñas no se guarden en texto plano, que los roles de usuario se respeten (por ejemplo, que un cajero no pueda entrar a las funciones del administrador) y que no se puedan inyectar comandos dañinos en los formularios.
- **Usabilidad:** Miden qué tan sencillo e intuitivo resulta usar el programa para una persona común. Revisan si los textos son claros, si los botones están en lugares lógicos, si los errores explican bien qué pasó y si personas con dificultades visuales pueden navegar bien.
- **Compatibilidad:** Comprueban que el sistema se vea y funcione bien en diferentes navegadores (Chrome, Firefox, Edge, Safari), en distintos tamaños de pantalla (computadoras, laptops, celulares) y en distintos sistemas operativos (Windows, Linux, macOS).
- **Confiabilidad y Recuperabilidad:** Evalúan si el sistema es estable ante fallas imprevistas (como si se va el internet o se apaga el servidor) y qué tan rápido se puede recuperar sin perder información importante.

Diferencias con las Pruebas Funcionales

La diferencia básica es que las funcionales revisan los requisitos del cliente sobre lo que debe hacer el sistema (entradas y salidas), mientras que las no funcionales revisan la calidad general de la aplicación en condiciones reales de uso. A continuación se presenta una tabla para comparar ambos tipos:

Aspecto	Pruebas Funcionales	Pruebas No Funcionales
Pregunta clave	¿Qué hace el sistema? (Revisa las funciones y reglas del negocio).	¿Cómo lo hace? (Revisa velocidad, seguridad y facilidad de uso).
Objetivo	Comprobar que a partir de ciertos datos de entrada se obtenga el resultado esperado.	Comprobar que el sistema sea estable, rápido y seguro bajo cualquier situación.
En qué se basan	Historias de usuario, casos de uso y requerimientos del cliente.	Tiempos de respuesta esperados, estándares de seguridad y experiencia de usuario.
Ejemplos	Iniciar sesión, guardar un producto, registrar una venta, calcular el IVA.	Medir cuánto tarda en cargar la página con 500 usuarios, probar si resiste ataques XSS.
Forma de ejecución	Se pueden hacer muy fácil de forma manual o con pruebas unitarias automáticas.	Casi siempre necesitan herramientas de simulación (como JMeter para crear usuarios virtuales).

1.2. Registro y Seguimiento de Defectos

¿Qué es un defecto en software? En palabras sencillas, un defecto (comúnmente llamado bug) es una imperfección o falla en el código, en la base de datos o en el diseño del software que hace que el programa no funcione como debería. En la ingeniería de software se suelen distinguir tres términos:

- **Error humano:** Es la equivocación que comete la persona (el programador o el diseñador), por ejemplo escribir mal una fórmula matemática o no validar que un campo no venga vacío.
- **Defecto o Bug:** Es el problema que queda guardado en el código fuente a causa del error humano.
- **Falla:** Es cuando el usuario o el tester ejecuta el programa, pasa por esa parte del código y el sistema hace algo incorrecto o se detiene.

Etapas del ciclo de vida de un defecto: Cuando se encuentra un bug, este pasa por varias etapas desde que se descubre hasta que se soluciona definitivamente:

- **1. Nuevo (New):** El tester o usuario nota el fallo, toma capturas de pantalla, anota los pasos exactos para provocarlo y crea el reporte.
- **2. Asignado (Assigned):** El encargado del proyecto revisa el reporte y se lo pasa al programador que conoce esa parte del sistema.
- **3. Abierto / En progreso (Open / In Progress):** El programador revisa el problema en su computadora, busca en qué línea de código está la causa y empieza a corregirlo.

- **4. Corregido (Fixed):** El desarrollador termina de hacer el cambio en el código y sube la nueva versión al entorno de pruebas.
- **5. En revisión (Pending Retest):** Se le avisa al equipo de pruebas que el cambio está listo para ser probado.
- **6. Re-prueba (Retest):** El tester vuelve a hacer exactamente los mismos pasos de antes para comprobar si el error ya desapareció.
- **7. Cerrado (Closed):** Si la prueba sale bien y ya no ocurre el fallo, el reporte se marca como resuelto y cerrado.
- **8. Reabierto (Reopened):** Si el problema sigue ocurriendo o si la corrección provocó otro error, se regresa el reporte al programador para que lo revise de nuevo.

Herramientas comunes para gestionar defectos: Para no perder el control de los errores y saber quién está arreglando qué cosa, los equipos usan programas especiales:

- **Jira:** Es la más utilizada en empresas de software. Permite organizar las tareas en tableros, asignar responsables, marcar la urgencia de cada bug y conectarse con GitHub o GitLab.
- **Bugzilla:** Es una herramienta gratuita y de código abierto bastante conocida. Es muy directa y se enfoca en registrar detalladamente cada error con su historial de comentarios.
- **TestLink:** Se enfoca principalmente en diseñar y organizar casos de prueba, pero se puede conectar con sistemas como Jira o Bugzilla para ligar las pruebas con los bugs encontrados.
- **GitHub Issues / GitLab Issues:** Vienen integrados directamente en la plataforma donde se guarda el código. Son muy cómodos porque se puede cerrar un reporte automáticamente escribiendo "fixes #12" en el commit.

1.3. Pruebas de Regresión

Objetivo y cuándo se aplican: El objetivo principal de las pruebas de regresión es asegurarse de que, al agregar una nueva función o arreglar un error, no se haya descompuesto algo que antes ya funcionaba bien. En el desarrollo de software es muy común que al tocar un archivo para mejorar una pantalla, sin querer se altere otra parte del sistema que compartía esa misma función.

Se aplican principalmente en las siguientes situaciones:

- **a) Después de corregir un error:** Para validar que la solución no haya roto otra cosa.
- **b) Al agregar funciones nuevas:** Para confirmar que lo nuevo conviva bien con lo viejo.
- **c) Al actualizar librerías o versiones:** Por ejemplo al cambiar de versión de Python o de base de datos.
- **d) Antes de publicar una nueva versión:** Como revisión final para tener seguridad de que todo el sistema sigue estable.

Estrategias y Herramientas Utilizadas

Cuando el proyecto se vuelve grande, repetir todas las pruebas a mano toma demasiado tiempo. Por eso se aplican estas estrategias:

- **Regresión total:** Consiste en volver a correr todas las pruebas del sistema. Es la opción más segura, pero requiere que las pruebas estén automatizadas para que la computadora las ejecute rápido.

- **Selección de casos:** En lugar de correr todo, se eligen únicamente las pruebas de las partes que pudieron haber sido afectadas por los cambios recientes.
- **Priorización de casos:** Se ordenan las pruebas para que primero se corran las más críticas del negocio (como cobrar o iniciar sesión) y después las secundarias.

Herramientas comunes: Se usan librerías de pruebas automatizadas como Pytest en Python o JUnit en Java, herramientas de navegador como Selenium o Playwright, y sistemas de integración continua como GitHub Actions para que las pruebas corran solas cada vez que alguien sube código.

Beneficios y Desafíos

- **Beneficios:** Dan mucha tranquilidad al equipo porque detectan fallos de inmediato antes de que lleguen a los usuarios finales, ahorran tiempo y dinero al evitar errores en producción, y permiten mejorar el código con confianza.
- **Desafíos:** Lleva tiempo escribir y mantener las pruebas actualizadas cuando las pantallas cambian, y a veces pueden existir pruebas inestables que fallen por problemas momentáneos de red o conexión.

2. Ejemplos Prácticos

A continuación se presentan tres casos sencillos y claros donde se aplica cada uno de los temas revisados:

Ejemplo 1: Sistema bancario sometido a pruebas de rendimiento

Imaginemos una aplicación de banca móvil de un banco que se prepara para el día de quincena. El equipo de sistemas necesita saber si los servidores van a aguantar a miles de personas consultando su saldo y haciendo transferencias al mismo tiempo entre las 12:00 y las 2:00 de la tarde.

Para probarlo sin arriesgar a los clientes reales, usan una herramienta llamada Apache JMeter que simula a 10,000 usuarios virtuales conectados al mismo tiempo. Al hacer la prueba, descubren que cuando se llega a 7,000 usuarios, la consulta de saldo pasa de tardar medio segundo a tardar 15 segundos, y la base de datos se satura porque se le acaban las conexiones disponibles. Gracias a esta prueba de rendimiento, el equipo optimiza las consultas y aumenta la capacidad de los servidores antes de la quincena, evitando que la aplicación bancaria se caiga en un día real.

Ejemplo 2: Un bug detectado y registrado en una herramienta de seguimiento (Jira)

En un sistema de punto de venta para una cafetería, el tester está probando el cierre de caja. Realiza una venta con tarjeta de $200 pesos, pero después el cliente pide cancelarla porque quiere pagar en efectivo. El cajero cancela la orden en la pantalla y genera el corte de caja. Sin embargo, en el resumen de caja el sistema sigue sumando los $200 pesos de la tarjeta como si estuvieran cobrados, provocando una diferencia de dinero.

El tester entra a Jira y crea un ticket con la siguiente información:

- **Título:** El corte de caja sigue sumando ventas canceladas con tarjeta.
- **Prioridad:** Alta (porque afecta el dinero y la contabilidad del negocio).
- **Pasos para reproducir:** 1. Cobrar $200 con tarjeta. 2. Cancelar la venta desde el historial. 3. Ir a corte de caja y ver totales.
- **Resultado esperado:** El total en tarjeta debe ser $0.00 pesos.
- **Resultado obtenido:** Aparecen $200.00 pesos cobrados en el reporte.

El ticket se asigna al programador, quien encuentra que a la consulta de la base de datos le faltaba filtrar las ventas canceladas. El programador hace el arreglo, sube la corrección, el tester repite la prueba, confirma que ya sale $0.00 y cierra el ticket en Jira como resuelto.

Ejemplo 3: Una nueva versión de software donde se ejecutan pruebas de regresión

El equipo del sistema de ventas decide lanzar la versión 2.0 para agregar una nueva función: permitir cupones de descuento del 10% por cumpleaños. El desarrollador programa la nueva lógica modificando la función general que calcula el total a pagar.

Antes de mandar esa versión a las tiendas, el sistema corre automáticamente en GitHub Actions una suite de pruebas de regresión que revisa todas las funciones viejas. Al correrlas,

salta una alerta roja: la prueba de ventas normales sin cupón falló porque la nueva función le sumaba cero al total pero le quitaba el cálculo del IVA a las ventas regulares.

Gracias a las pruebas de regresión, el equipo detectó el fallo en menos de un minuto y evitó publicar una versión que hubiera cobrado mal los impuestos en todas las sucursales. El programador ajustó el código, las pruebas volvieron a pasar todas en verde y se pudo liberar la actualización con total tranquilidad.

3. Análisis Reflexivo

Importancia de las pruebas no funcionales frente a las funcionales: ¿Por qué las pruebas no funcionales son tan relevantes como las funcionales?

Ambos tipos de pruebas son igual de importantes porque se complementan. Un programa puede tener todas sus funciones bien hechas (hacer sumas exactas, guardar en la base de datos o emitir recibos), pero si tarda demasiado tiempo en responder, si se cae apenas entran varios usuarios o si no cuida la privacidad de las contraseñas, en la práctica el software no sirve. De nada le sirve a un negocio tener un sistema que calcula bien si los clientes se van molestos por la lentitud o si sufren un robo de datos. Por eso la velocidad, la seguridad y la facilidad de uso son tan indispensables como el funcionamiento de los botones.

El seguimiento de defectos en la mejora continua: ¿Qué papel juega el seguimiento de defectos en la mejora continua del software?

Llevar un registro ordenado de los defectos no solo sirve para saber qué cosas faltan por arreglar en la semana, sino que ayuda a que el equipo aprenda de sus propios errores. Al ver el historial de fallas, el equipo puede darse cuenta de qué módulos del sistema suelen fallar más seguido, por qué ocurren esas equivocaciones y qué partes necesitan rediseñarse con más cuidado. Además, cada vez que se arregla un bug importante, se puede crear una prueba automática para asegurar que ese mismo error nunca vuelva a aparecer. Así, el registro de defectos se convierte en una herramienta clave para que el software sea cada vez más maduro, estable y confiable.

4. Referencias (Formato IEEE)

[1] IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008, IEEE Computer Society, Piscataway, NJ, USA, 2008.

[2] ISO/IEC Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, ISO/IEC 25010:2011, International Organization for Standardization, Ginebra, Suiza, 2011.

[3] I. Sommerville, Software Engineering, 10.ª ed. Boston, MA, USA: Pearson Education, 2016.

[4] R. S. Pressman y B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9.ª ed. Nueva York, NY, USA: McGraw-Hill Education, 2020.